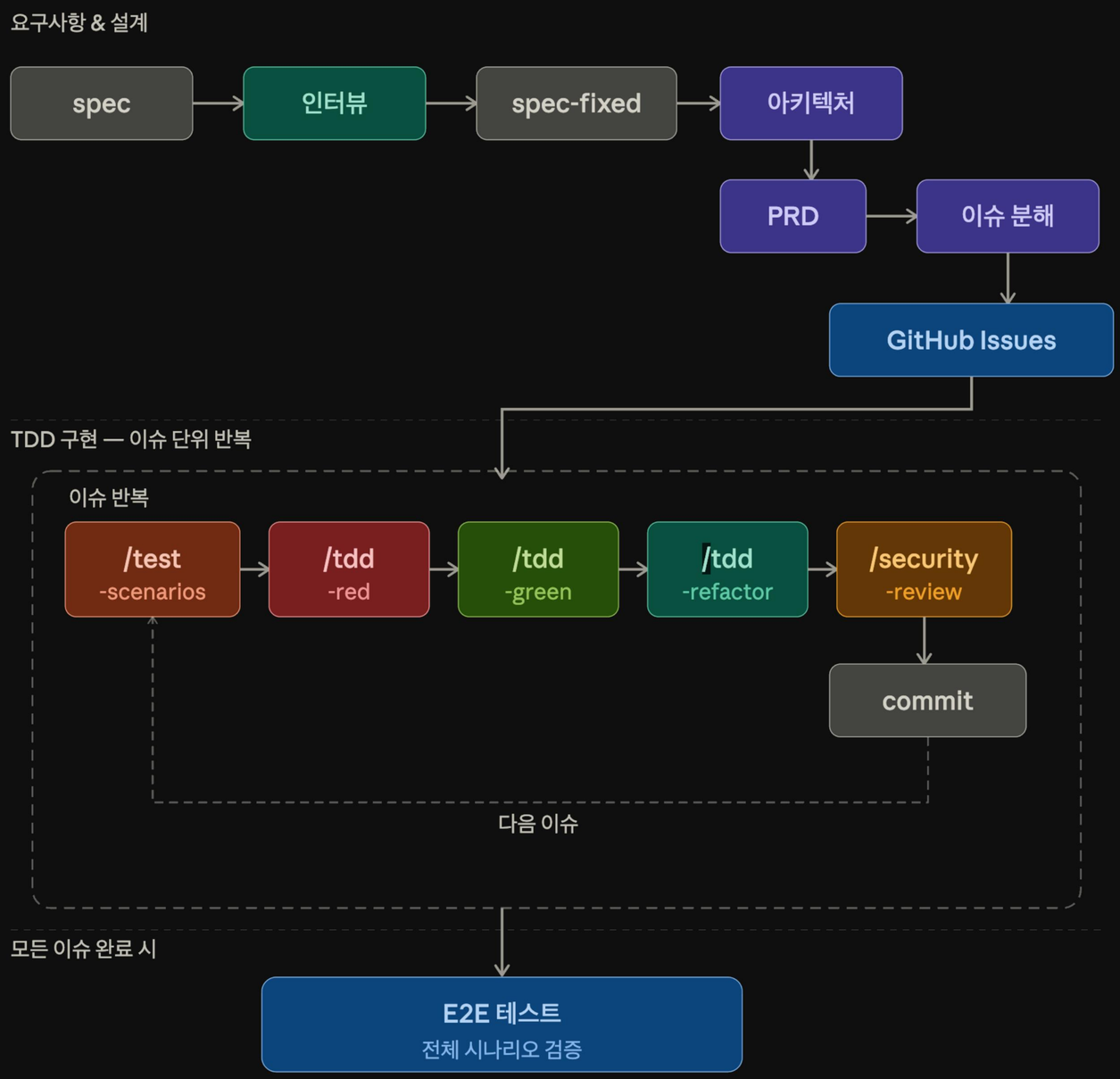
4강. 요구사항 인터뷰

spec-fixed로 결정 확정하기


요구사항 & 설계
spec
인터뷰
spec-fixed
아키텍처
PRD
이슈 분해
GitHub Issues
TDD 구현 — 이슈 단위 반복
이슈 반복
/test
-scenarios
/tdd
-red
/tdd
-green
/tdd
-refactor
/security
-review
commit
다음 이슈
모든 이슈 완료 시
E2E 테스트
전체 시나리오 검증

"태그 기능 만들어줘"

AI에게 기능을 한 마디로 던지면 AI가 **알아서 결정**한다

(지 마음대로..)

별도 테이블? 배열? 최대 개수? 대소문자?

잘못된 결정 위에 쌓이는 코드

발견 시점	상대적 수정 비용
요구사항 단계	**1x**
설계 단계	5x
구현 단계	10x
테스트 단계	15x
배포 후	**100x**

요구사항 1시간 = 배포 후 100시간 절감

지금 여기 있습니다

현재: 요구사항 인터뷰 → spec-fixed

완료: 환경 설정 (CLAUDE.md · Hook · husky · commitlint)

다음: 디자인 시스템 구축



이번 강의 포인트

AI 인터뷰

모호한 요청을 구체적 명세로 변환할 수 있다

수정 비용 최소화

요구사항 단계 확정이 비용을 최소화하는 이유를 수치로 설명할 수 있다

spec-original / spec-fixed

분리하여 요구사항 변경의 추적성을 확보할 수 있다

요구사항 엔지니어링

코드는 **"어떻게"**의 영역

요구사항은 **"무엇을"**의 영역

AI 시대 — 코드가 빠르게 쌓인다 → **방향이 더 중요**

기능 요구사항 — 시스템이 무엇을 해야 하는가

비기능 요구사항 — 어떤 조건을 만족해야 하는가

AI 인터뷰 — 역할을 뒤집는다

개발자가 묻는 게 아니라 **AI가 묻게 한다** (질문해보렴)

```
이 기능 정의서를 읽고, 코드베이스를 탐색해서
구현 전에 확인이 필요한 것들을 질문해라.
한 번에 하나씩 질문하고, 추천 방식도 포함해라.
```

AI는 수만 개 프로젝트의 실패 패턴을 학습 (나보다 낫다..) — 빠진 엣지 케이스를 짚어낸다

3가지 프롬프트 패턴

"한 번에 하나씩"

집중도 유지, 깊이 있는 답변 유도

"추천 방식도 이유와 함께"

판단 근거 제공, 결정 속도 향상

"코드베이스 먼저 탐색"

현실적 질문, 기존 패턴과 충돌 방지

spec-original vs spec-fixed

파일	역할
spec-original.md	원본 기능 정의서
spec-fixed.md	인터뷰 후 확정된 명세

추적성(Traceability): *"왜 태그 최대 개수가 5개인가?"에 답할 수 있어야 한다*

변경 이유가 기록되어야 — 나중에 제약을 바꿔도 되는지 판단 가능

실습

AI 인터뷰 세션

spec-original.md 준비 → 인터뷰 프롬프트 실행

인터뷰 진행

한 번에 하나씩 답변 → 결정사항 확정

spec-fixed.md 생성

인터뷰 결과 + 용어 정의 섹션 포함

커밋

`docs(tag): spec-original 및 spec-fixed 작성`